V3 Redesign: Closing Two Loopholes in the Guard/Rose Gödel II Formalization

Abstract

The V2 formalization of Gödel II in the Guard/Rose equational system (see `Guard/archive/guard-godelII.tex`) proves a strictly weaker theorem than the colloquial statement: its object-level proof evaluator thFunTFor admits *two* encoding loopholes that make *every* consistency sentence refutable by pure computation, independent of the object theory's actual axiomatic content. This V3 redesign adds a single new primitive, $\mathrm{RecP} : \mathrm{Fun}_2 \to \mathrm{Fun}_2$, and restructures the evaluator into a hypothesis-aware thmT, closing both loopholes and producing a genuine internal Gödel II.

1 The two V2 loopholes

1.1 `mkProofEAny` (the encoding loophole)

V2's Theorem 14 takes *any* equation H (the ambient hypothesis) and produces a proof-encoding $\ulcorner H \urcorner$ such that $\mathrm{thFunTFor}(\ulcorner H \urcorner) = \widehat{\mathrm{codeEqn}}\, H$ — not because H is actually derivable from itself, but because the encoding function `mkProofEAny` constructs a *fake tree* whose recursive evaluation ignores H's truth and just echoes H's coded form. This collapses the constructive statement "$H \vdash \mathrm{Con}_H \Rightarrow H \vdash \bot$" to a pure computation: ANY equation H satisfies the premise's syntactic shape, so ANY equation refutes Con_H.

1.2 `case19` (the middle-term loophole)

V2's thFunTFor handles the `ruleTrans` case by emitting $\mathrm{Pair}(\mathrm{left}(sp_1), \mathrm{right}(sp_2))$ without checking that $\mathrm{right}(sp_1) = \mathrm{left}(sp_2)$. This means

$$\mathrm{trans}(\mathrm{refl}\,\mathbf{t}, \mathrm{refl}\,\mathbf{f})$$

has codeEqn-evaluation result $\widehat{\mathrm{codeEqn}(\mathbf{t} = \mathbf{f})}$ — a fake encoding of the bottom equation built from genuine refl-proofs. Combined with `mkProofEAny`, this compounds the first loophole.

2 The V3 fix

2.1 New primitive: RecP

We extend the functor syntax with

$$\mathrm{RecP} : \mathrm{Fun}_2 \to \mathrm{Fun}_2, \qquad \mathrm{RecP}(s)\, p\, O = O, \quad \mathrm{RecP}(s)\, p\, \mathrm{Pair}(a, b) = s\, \mathrm{Pair}(p, \mathrm{Pair}(a, b))\, \mathrm{Pair}(\mathrm{RecP}(s)\, p\, a,\ \mathrm{RecP}(s)$$

This is a *parameterised* tree-recursion: the step function s receives the RecP-argument p packed alongside the current node $\mathrm{Pair}(a, b)$, giving closed-form access to p without embedding it as a free variable inside s's structure.

From RecP we build

$$\mathrm{substOp} : \mathrm{Fun}_2 \qquad \mathrm{substOp} = \mathrm{RecP}(\mathrm{stepSubstP})$$

with a step function that inspects the current node's tag and, when it sees tagVar, substitutes according to the parameter pair $\mathrm{Pair}(t_C, x_C)$. The load-bearing correctness lemma is

$$\mathrm{substOp}\,\mathrm{Pair}(\widehat{\mathrm{code}\,t},\ \widehat{\mathrm{natCode}\,x})\,\widehat{\mathrm{code}\,l} = \mathrm{code}(\widehat{\mathrm{subst}}\,x\,t\,l).$$

2.2 Case 19: middle-term matching

$$\mathrm{case19V3} = \mathrm{Fan}\big(\mathrm{Fan}\,\mathrm{recs}_{\mathrm{AR}}\,\mathrm{recs}_{\mathrm{BL}}\,\mathrm{TreeEq}\big)\ \big(\mathrm{Fan}\,(\mathrm{Fan}\,\mathrm{recs}_{\mathrm{AL}}\,\mathrm{recs}_{\mathrm{BR}}\,\mathrm{Pair})\ (\mathrm{kF}_2\,O)\ \mathrm{Pair}\big)\ \mathrm{IfLf}$$

This inserts a TreeEq check between $\mathrm{right}(sp_1)$ and $\mathrm{left}(sp_2)$: on match, emit $\mathrm{Pair}(\mathrm{left}(sp_1), \mathrm{right}(sp_2))$; on mismatch, emit the sentinel O. The attempted fake $\mathrm{trans}(\mathrm{refl}\,\mathbf{t}, \mathrm{refl}\,\mathbf{f})$ no longer typechecks in the V3 system because the middle terms fail to unify.

2.3 Case 23: dynamic ruleInst via substOp

$$\mathrm{case23V3} = \mathrm{Fan}\ (\mathrm{Fan}\,\mathrm{origA}\,\mathrm{recs}_{\mathrm{BL}}\,\mathrm{substOp})\ (\mathrm{Fan}\,\mathrm{origA}\,\mathrm{recs}_{\mathrm{BR}}\,\mathrm{substOp})\ \mathrm{Pair}$$

V2's `case23` applied `substTFor` (a closed Fun_1 with free variables v_{11}, v_{12} inside it) to each sub-equation side, and later *ruleInst-bound* v_{11}, v_{12} at proof-construction time. V3's `case23V3` evaluates substOp at the Fun_2-level with the substitution parameters already packed into the `Pair` argument — no var-capture.

2.4 Hypothesis check: `case26`

At the dispatch bottom we add a new tag n_{26} with

$$\mathrm{case26}(h) = \mathrm{Fan}\,(\mathrm{Fan}\,(\mathrm{Lift}\,\mathrm{Snd})\,(\mathrm{kF}_2\,h)\,\mathrm{TreeEq})\ (\mathrm{Fan}\,(\mathrm{kF}_2\,h)\,(\mathrm{kF}_2\,O)\,\mathrm{Pair})\ \mathrm{IfLf}$$

and the ruleHyp encoding is redefined as $\mathrm{encHyp}(h) = \mathrm{nd}(\mathrm{natCode}\,n_{26},\ h)$. The encoding is accepted only when the body literally equals the ambient hypothesis code h, so `mkProofEAny` no longer passes: any encoding of `ruleHyp` at a fake-hypothesis H' now evaluates to the sentinel O under $\mathrm{thmT}(\widehat{\mathrm{codeEqn}}\,H)$.

3 What thmT means

$\mathrm{thmT} : \mathrm{Term} \to \mathrm{Fun}_1$ is the object-level *proof evaluator* — a decoder from encoded proof-trees back to reified equation-codes. For any hypothesis code $h = \widehat{\ulcorner H \urcorner}$, the specialization $\mathrm{thmT}(h) : \mathrm{Fun}_1$ is a closed primitive recursive tree function. Applied to a Term x (the encoded proof):

- $x = O$: result is O (the sentinel).
- $x = \mathrm{Pair}(\mathrm{natCode}\,n, b)$ for some $n \in \{0, \ldots, 28\}$: dispatch to the case for rule n. Each case_n extracts the relevant sub-data from b and emits the reified codeEqn of that rule's conclusion.
- $n = 26$ (`ruleHyp`): check $b \equiv h$ via TreeEq; on match return h, on mismatch return the sentinel O.
- Other shapes: sentinel O.

The load-bearing invariant is soundness of the encoding (Theorem 14 of V3, `thm14EV3`):

Theorem 1 (Encoding soundness)**.** *For every* $d : \mathrm{Deriv}\, H\, A$*, there is a Term* $\mathrm{enc}(d)$ *(computed by* `thm14EV3` d*) such that for every Deriv-hypothesis* hyp*:*

$$\mathrm{hyp} \vdash \ \mathrm{thmT}(\widehat{\ulcorner H \urcorner})(\mathrm{enc}(d)) = \widehat{\ulcorner A \urcorner}.$$

This is a *decoder*, not a boolean check. $\mathrm{thmT}(\widehat{\ulcorner H \urcorner})(x)$ always produces some equation-code or the sentinel O; it never produces a truth-value. Also note: the result Term is a *reified* codeEqn, whose shape is always of the form Pair $* *$ — disjoint from the sentinel O. So $\mathrm{thmT}(\widehat{\ulcorner H \urcorner})(x) = O$ is the unambiguous "invalid encoding" outcome; a non-sentinel value is some claimed conclusion code.

V3 does *not* prove the converse (decoder-faithfulness: every x with $\mathrm{thmT}(\widehat{\ulcorner H \urcorner})(x) = \widehat{\ulcorner A \urcorner}$ comes from a genuine $\mathrm{Deriv}\, H\, A$). The V3 shape-check refinements (case19V3, case26) only rule out the specific V2 loopholes; full faithfulness is a stronger claim we do not need.

4 How consistency is represented

The V3 internal consistency statement for the Gödel sentence gs_{V3} is the single *Equation*:

$$\mathrm{conSentenceV3} \ = \ \mathrm{TreeEq}\big(\, \mathrm{thmT}(\widehat{cGS})(\mathrm{var}\, 0), \ \ \widehat{\mathrm{codeEqn}(\mathbf{t} = \mathbf{f})}\, \big) \ = \ \mathbf{f}.$$

Unfolding: the equation has one free variable (the proof-slot $\mathrm{var}\, 0$). Derivation rules allow ruleInst at this variable, so assuming conSentenceV3 as an axiom is equivalent to assuming the *universal* "for every candidate encoding x, $\mathrm{TreeEq}(\mathrm{thmT}(\widehat{cGS})(x), \widehat{\ulcorner \bot \urcorner}) = \mathbf{f}$".

Reading the inner equation: for each x, the thmT-evaluator decodes x as an alleged proof-encoding with hypothesis gs_{V3}; the TreeEq compares the decoder's output against $\widehat{\mathrm{codeEqn}(\bot)}$; the whole sentence asserts this comparison yields $\mathbf{f}$ for every x. Equivalently: *no x encodes a* gs_{V3}*-derivation of* $\bot$. That is $\mathrm{Con}_{\mathrm{gs}_{V3}}$ reflected inside the object language via the V3 decoder.

There are two complementary layers:

- **Meta-level (Agda) consistency**: $\mathrm{Consistent}\, \mathrm{gs}_{V3} := \mathrm{Deriv}\, \mathrm{gs}_{V3}\, (\mathbf{t} = \mathbf{f}) \to \bot$. An Agda type, not a formal statement of the object theory.
- **Object-level (internal) consistency**: conSentenceV3 above, a single Equation in the object language, expressible because the equational system is rich enough to represent its own decoder thmT.

5 The V3 Gödel I argument

The V2 diagonal $\mathrm{cstf}\, (\widehat{\mathrm{templateCode}}) = \widehat{cGS}$ translates to V3 via substOp — `diagFTargetV3` in `Guard/GodelIV3.agda`. The key wrinkle: since thmT is hypothesis-specific, the V2 trick of substituting a single variable v_1 with $\widehat{\mathrm{templateCode}}$ in the template doesn't suffice: after substitution, the sentence's embedded thmT-hCode becomes $\widehat{\mathrm{templateCode}}$, whereas $\mathrm{thm}_{14\mathrm{E}}\mathrm{V3}$'s correctness witness produces $\mathrm{thmT}(\widehat{cGS})$.

Fix: use *two* free variables in the template — v_1 for the self-code (substituted in the template) and v_2 for thmT's hCode (kept free until Deriv-level ruleInst). The Gödel I proof chain then threads four ruleInst's ($v_2 \mapsto \widehat{cGS}$, $v'_{11} \mapsto \widehat{crTC}$, $v'_{12} \mapsto \widehat{\mathrm{natCode}}\, v_1$, $0 \mapsto enc$) to bring the sentence's embedded thmT-hCode into alignment with $\mathrm{corr}\, pe$'s witness. Both sides of the sentence's TreeEq collapse to $\widehat{cGS}$; `treeEqSelf` closes the diagonal.

Theorem 2 (Gödel I on V3 — `godelIDerivV3` in `GodelIV3.agda`)**.** *For the V3 Gödel sentence* gs_{V3}*:*

$$\mathrm{gs}_{V3} \vdash \mathbf{t} = \mathbf{f}.$$

6 The V3 Gödel II statements

The same argument furnishes three progressively-sharper formulations.

Theorem 3 (Gödel II, positive form — `godelIIPositive`)**.**

$$\mathrm{gs}_{V3} \vdash \mathrm{conSentenceV3} \implies \mathrm{gs}_{V3} \vdash \mathbf{t} = \mathbf{f}.$$

Theorem 4 (Gödel II, contrapositive — `godelIIV3`)**.** *If* gs_{V3} *is meta-consistent, then* $\mathrm{gs}_{V3} \nvdash \mathrm{conSentenceV3}$*.*

Theorem 5 (Gödel II, sharpest — `godelIIConAlone`)**.** $\mathrm{conSentenceV3} \vdash \mathbf{t} = \mathbf{f}$*.*

Theorem **??** is a one-line consequence of Theorem **??** applied at hyp = conSentenceV3 via ruleHyp: the internal consistency sentence, taken as the sole axiom, proves $\bot$.

Proof sketch

Necessitation of `godelIDerivV3` gives a `Prov3`-witness $enc_\bot$ with $\mathrm{thmT}(\widehat{cGS})(enc_\bot) = \mathrm{code}\widehat{\mathrm{Eqn}(\mathbf{t} = \mathbf{f})}$ — crucially, this equation is derivable in any Deriv-hypothesis (polymorphic in hyp). Plugging $enc_\bot$ into conSentenceV3 via ruleInst and rewriting the thmT-application using corr *provBot*, both arguments of the internal TreeEq become $\mathrm{code}\widehat{\mathrm{Eqn}(\mathbf{t} = \mathbf{f})}$; treeEqSelf then contradicts the sentence's claim that this should equal $\mathbf{f}$.

Why the sharpest form is the natural one

Classical Gödel II over PA cannot construct a concrete $enc_\bot$ — PA is (we believe) consistent, so no such $\bot$-proof exists. Here, gs_{V3} is *deliberately* inconsistent by the diagonal construction (Theorem **??** below, in the Gödel I section), so $enc_\bot$ exists and the base equational system has enough computational content to verify it. conSentenceV3 asserts no such witness exists; the system concretely refutes that claim. This is the sharpest positive reading of Gödel II: the internal consistency sentence is refutable *in the base system extended with it as an axiom*, because the base system can exhibit a $\bot$-proof whose validity the sentence asserts to be impossible.

7 Negative acceptance test

`Guard/GodelIITestV3.agda` documents that the V2-style attack $\mathrm{thm}_{14\mathrm{E}}\mathrm{V3Trans}(\mathrm{refl}\,\mathbf{t})(\mathrm{refl}\,\mathbf{f})$ is rejected at type-checking time: Agda reports $\mathbf{f} = \mathrm{Pair}(O, O) \neq O = \mathbf{t}$ when unifying the two middle terms. V3 has no `mkProofEAny`-like escape hatch, so the type system structurally rules out the V2 loopholes.

8 Files and sizes

All files typecheck under `--safe --without-K --exact-split` with the `agda-2.9.0` binary. Clean rebuild of `Guard/GodelIIV3.agda` (the top of the V3 chain) is ∼7 s.

- `Guard/Term.agda`, `Guard/Step.agda` — RecP primitive and its axioms `axRecPLf`, `axRecPNd`.
- `Guard/SubstOp.agda` — substOp + `substOpEquiv` + `substOpCorrect`.
- `Guard/ThFunTForV3.agda` — the hypothesis-aware dispatcher, including `case19V3`, `case23V3`, `case26`, `case27`, `case28`.
- `Guard/Thm14EV3.agda` — `thm14EV3` (total, 29 Deriv constructors) and the per-rule encodings.
- `Guard/SubstTForPrecompV3.agda` — V3 diagonal machinery (templateCode$_{V3}$, cGS$_{V3}$, etc.) plus the fused `fullyInstGS` substEq lemma.
- `Guard/GodelIV3.agda` — `godelIDerivV3`, `diagFTargetV3`.
- `Guard/ProvV3.agda` — `Prov3`, `necessitation`, `diagContradict`.
- `Guard/GodelIIV3.agda` — `provBot`, `conToBotV3`, `godelIIV3`.
- `Guard/GodelIITestV3.agda` — negative acceptance test.